A. Additional model details

We train causal, 2-layer attention-only transformer models, as used by prior work (Olsson et al., 2022), to study induction circuit formation. Similar to prior work (Chan et al., 2022; Singh et al., 2023), we only train to minimize the cross entropy loss on the last token – the predicted label for the query exemplar. We use RoPE (Su et al., 2021) for positional encoding and a model dimension of 64, with 8 heads per layer. As is common for transformers outside of interpretability work, we are able to use LayerNorm (Ba et al., 2016) when reading in from the residual stream to an attention block – LayerNorm is often excluded in interpretability work because it complicates standard analyses (Elhage et al., 2021), but our causal framework works by modifying activations instead of inspecting weights or assuming linearity, circumventing the issues.

For optimization, we use Adam (Kingma & Ba, 2015) from the `optax` library (DeepMind et al., 2020), with parameters $\beta_1 = 0.9, \beta_2 = 0.999$. We use a constant learning rate of $1e-5$ and a batch size of 32 sequences.

B. Computational cost to run experiments

All experiments were run on a 2018 MacBookPro with a 6-Core Intel Core i7 processor and 16GB of RAM. Standard training runs (e.g., Figure 3) took $\approx$ 7 minutes each to complete (for $1e6$ sequences seen). Clamped runs were more expensive per iteration (as some clamps require multiple forward passes), but overall took even less time since they converge in fewer sequences, note x-axis in Figure 5. This speed was possible due to the use of `jax.jit`, without which each training run would take $\approx$ 330 minutes on the same hardware. Though PyTorch has recently added `torch.compile` to impart similar functionality (PyTorch, 2022), interpretability frameworks based in PyTorch are currently not compatible with `torch.compile` due to their use of hooks (Cooney, 2023). Our JAX-based framework is natively compatible with `jax.jit`, as the `call_with_all_aux` formalism uses PyTrees, which could possibly mean speed-ups for other interpretability researchers (as it did for us).

C. Dataset size calculation

For a dataset of C classes, E exemplars, L labels, and assuming a fraction F of relabelings being used for training, our total dataset size is

$$\binom{C}{2} \cdot F \cdot \binom{L}{2} \cdot E^3 \cdot 2 \cdot 2 \cdot 2 = 2FE^3C(C-1)L(L-1) \Rightarrow 1.6C(C-1)L(L-1)$$

for our settings where $E = 1$ and $F = 0.8$ are fixed.

D. Additional details on clamping

D.1. Implementation details of clamping for induction head formation

In Section 4.2, we detail the methodology and inspiration for clamping various subcircuits. In this subsection, we detail the specifics of implementation of each of the clamped steps.

D.1.1. Clamping weights vs activations: Complications due to routing

We first demonstrate that clamping isn't as easy as just fixing weights during training. Recall that we want to clamp computation steps (Section 4.1) that comprise an induction circuit. There are many ways (in terms of weights) the network can implement some of these computation steps, as many of them involve reading from and/or writing into a subspace of the residual stream (e.g., Steps 2 and 3). Arbitrary rotations could be applied to these subspaces.[18] The specific subspaces used appear to be initialization dependent, as shown in Figure 7.

We consider fixing the weights of Layer 1 to match those of networks pre-trained on our task, but from different initializations. Specifically, we train networks on our task from different initialization seeds (which we will refer to as seeds[19] 5, 6, 7). All networks learn induction circuits, with multiple previous token heads appearing in Layer 1. We then train a network initialized from seed 5 with the residual stream after Layer 1 clamped to the residual stream after Layer 1 of one of the fully trained networks (equivalent to fixing all the weights up to this point). Note that these activations are fixed for a given input,

[18] This has been referred to the as *identification failures* by some (Shalizi, 2024).

[19] These values are arbitrary and are the first three we tried. We didn't use seed 0 as this was the seed for our data sampler.

so gradients are only flowing to Layer 2 of the model (as desired, since we are clamping Layer 1 to be an externally "perfect" Layer 1 based on weights). The clamped Layer 1 is theoretically perfectly performing Steps 1 and 2. However, we can see that there is substantial variance when the weights used are from the same seed as that used for the Layer 2 initialization (red, Figure 7) versus when the weights used are from a different seed (blue or green, Figure 7). These results point to the importance of Step 3, routing, and tell us that fixing weights may not be the best way to clamp subcircuits as it may be confounded by the specific methodology used (specifically, which network the weights come from).

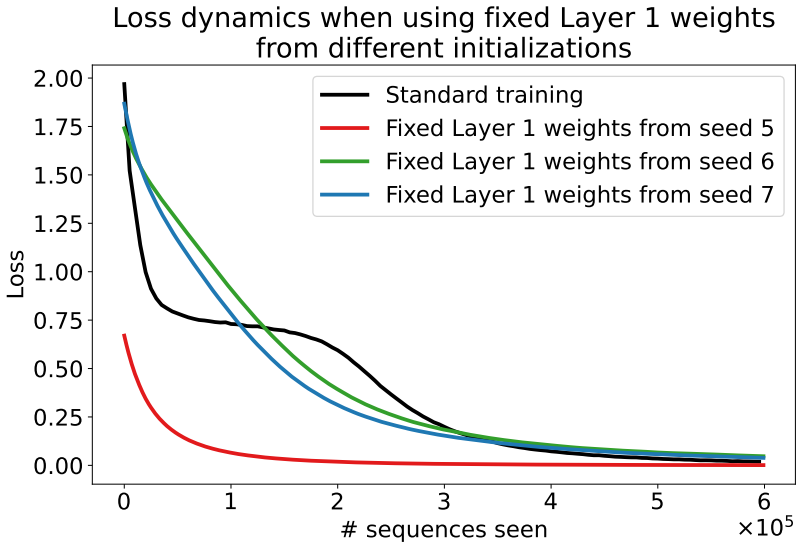

Figure 7. Loss curves when fixing Layer 1 weights to be those found at the end of training with various seeds. We see that when using the Layer 1 weights from the end of training off of the same initialization seed (seed 5) learning is way quicker, indicating that specific implementations of computations 1 and 2 in Section 4.1 are tightly coupled to initialization. Furthermore, these results suggest that fixing weights is not the ideal form of clamping as it is very susceptible to correlations between the chosen weights and network initialization.

D.1.2. Activation clamps that we used

Building off this insight, we instead implement our clamps at the activation level. While this does restrict the set of useful clamps we may apply, all the clamps used in Section 4.2 can be expressed at the activation level. We detail each clamp below:

Clamping Step 1 (PT-attend) We fix the attention pattern of a Layer 1 head to always attend to its previous token (with post-softmax value 1), corresponding to clamping Step 1 from Section 4.1. In Section 4.2, we clamped Layer 1 Head 2's attention pattern. In Figure 8, we show the effect of this clamp applied to different sets of Layer 1 heads. We find that overall there is little difference, unless all heads are clamped, in which case learning is severely slowed. We hypothesize that some Layer 1 heads speed up learning of copy circuits (which is consistent with the findings of V-composition in Appendix E) – by forcing all Layer 1 heads to have the previous token attention pattern, we slow this mechanism. We didn't investigate this further, however, as this was not the focus of our work, and instead used a clamp on just one previous token head for our main experiments.

Clamping Steps 1, 2 (layer 1) To implement this clamp while taking into account the concerns related to routing from Appendix D.1.1, we consider how Layer 1 outputs are used by Layer 2. We utilize two forward passes through the network. In the first, we set the input to Layer 2 to be equivalent to the output from previous token heads and perfect query routing. For example, for the sequence "A 0 B 1 A", this would look like "A A 0 B A", where we note that the first token cannot attend to a previous token (hence it still has 'A'), the next 3 tokens attend to previous tokens (so '0 B 1' becomes 'A 0 B'), and the query token still routes from the input ('A'). We directly pass the embeddings of these tokens to Layer 2, and cache the *patterns*. Then, we do a second forward pass using these patterns, except now we clamp the input to Layer 2 to match the input to the network ("A 0 B 1 A") which is what the value routing should be reading from. We note that this clamp is making the task a bit easier for the network as the vectors that Routing QK needs to read from and Routing V needs to read from are not additively superimposed, but these routing steps need to learn to read from the correct subspaces. We opted for

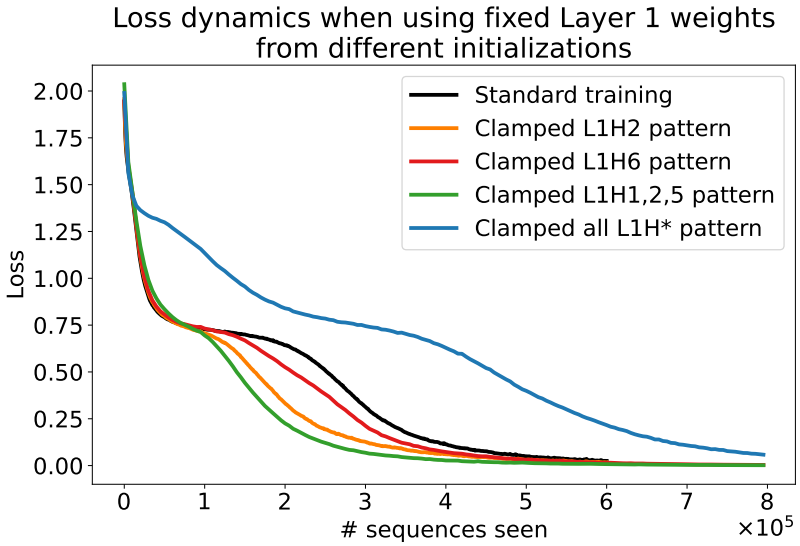

Figure 8. Effect of different implementations of the previous token attention pattern clamp (clamping computation Step 1, using the terminology of Section 4.1). We see that most versions of the clamp have little effect on dynamics (just slight left or right shifts in the phase change), with the exception of clamping all heads in Layer 1. We suspect this may be due to some Layer 1 heads V-composing with Layer 2 heads to make copy circuits easier to learn.

this option as it makes it even more telling that there are some subcircuit interactions in the remaining steps after applying this clamp.

Clamping Steps 1, 2, 3qk, 4 (match) We implement this by fixing the attention pattern of a Layer 2 head (we picked Layer 2 Head 3 as described in Section 4.2) to be that of a perfect induction head – 1 to the correct label token and 0 to all other tokens. The aim of this clamp is to make sure the correct label is matched to (thus isolating the copy operation), which this implementation suffices to do.

Clamping Steps 3v, 5 (copy) We implement this clamp by considering the attention logits to the two labels in-context at a given induction head (we picked Layer 2 Head 3 as described in Section 4.2) and setting the output logits to them. For example, on the sequence "A 0 B 1 A", the attention logits from the query token 'A' to the label tokens '0' and '1' would be the output logits for labels 0 and 1. The other output logits are set to $-1e9$. The aim of this clamp is to make sure labels are perfectly copied (thus isolating the match operation), which this implementation suffices to do.

Clamping Steps 1, 2, 3v, 5 We implement this clamp by combining the clamp for Steps 1, 2 (layer 1), and the clamp for Steps 3v, 5 (copy). As each of these clamps are disjoint, it's easy to apply both, serving the inteded purpose.

D.2. Contrasting *clamping* to progress measures

One prevailing current method in mechanistic interpretability for understanding training dynamics is to use *progress measures*. Progress measures track quantities through the normal training process, often using causal ablations of intermediate network checkpoints.[20] Despite this causal nature per checkpoint, we note that progress measures are actually *correlational* metrics, as the learning dynamics of the quantities they track may or may not be influencing the learning dynamics of the overall network. As such, we find our clamping method (Section 4.2) to be more useful for uncovering underlying subcircuits.

We first illustrate this with the toy model explored in Section 4.2. To track the evolution of "subcircuit" $\mathbf{a}$, we might consider various progress measures. The first problem we run into is that $\mathbf{a}, \mathbf{b}, \mathbf{c}$ are only specified up to scalars – if $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ is a solution, so is $(-\mathbf{a}, -\mathbf{b}, \mathbf{c})$. We find this to be an interesting toy analog to the rotations problem described in Appendix D.1.1. Given this, we might consider tracking the squared cosine distance between $\mathbf{a}$ and $\mathbf{a}^{true}$. Another progress measure we

[20] Our results in Figure 3b and Figure 12 are examples of progress measures, with the latter involving a causal ablation per checkpoint.